```
from google.colab import files
uploaded = files.upload()  # Upload ner_dataset.csv
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving ner_dataset.csv to ner_dataset.csv
```

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from torch.nn.utils.rnn import pad_sequence
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cpu
```

```
# Load and forward-fill sentence ids
data = pd.read_csv("ner_dataset.csv", encoding="latin1")
data = data.ffill()

# Basic sanity checks
required_cols = {"Sentence #", "Word", "Tag"}
assert required_cols.issubset(set(data.columns)), f"Dataset must contain columns: {required_cols}"

print("Rows:", len(data))
print("Unique words:", data["Word"].nunique())
print("Unique tags:", data["Tag"].nunique())
```

```
Rows: 1048575
Unique words: 35177
Unique tags: 17
```

```
words = list(data["Word"].astype(str).unique())
tags = list(data["Tag"].astype(str).unique())

# Add special tokens
PAD_WORD = "ENDPAD"
PAD_TAG = "O"

if PAD_WORD not in words:
    words.append(PAD_WORD)

if PAD_TAG not in tags:
    tags.append(PAD_TAG)

word2idx = {w: i+1 for i, w in enumerate(words)}  # start from 1
tag2idx  = {t: i for i, t in enumerate(tags)}
idx2tag  = {i: t for t, i in tag2idx.items()}

vocab_size = len(word2idx) + 1  # +1 safety
num_tags   = len(tag2idx)

print("Vocab size:", vocab_size)
print("Num tags:", num_tags)
```

```
Vocab size: 35179
Num tags: 17
```

```
class SentenceGetter:
    def __init__(self, df):
        self.grouped = df.groupby("Sentence #", group_keys=False).apply(
            lambda s: [(str(w), str(t)) for w, t in zip(s["Word"], s["Tag"])]
        )
        self.sentences = list(self.grouped)

getter = SentenceGetter(data)
sentences = getter.sentences

print("Total sentences:", len(sentences))
print("Example sentence:", sentences[0][:10])
```

```
Total sentences: 47959
Example sentence: [('Thousands', 'O'), ('of', 'O'), ('demonstrators', 'O'), ('have', 'O'), ('marched', 'O'), ('through', 'O'
/tmp/ipykernel_360/2742093026.py:3: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavi
  self.grouped = df.groupby("Sentence #", group_keys=False).apply(
```

```
X = [[word2idx[w] for w, t in s] for s in sentences]
y = [[tag2idx[t] for w, t in s] for s in sentences]

lengths = [len(s) for s in X]
print("Max sentence length in corpus:", max(lengths))
```

```
Max sentence length in corpus: 104
```

```
max_len = 50  # common choice

X_pad = pad_sequence([torch.tensor(seq) for seq in X],
                     batch_first=True,
                     padding_value=word2idx[PAD_WORD])

y_pad = pad_sequence([torch.tensor(seq) for seq in y],
                     batch_first=True,
                     padding_value=tag2idx[PAD_TAG])

# truncate to max_len
X_pad = X_pad[:, :max_len]
y_pad = y_pad[:, :max_len]

print("X_pad shape:", X_pad.shape)
print("y_pad shape:", y_pad.shape)
```

```
X_pad shape: torch.Size([47959, 50])
y_pad shape: torch.Size([47959, 50])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_pad, y_pad, test_size=0.2, random_state=42
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

```
Train size: 38367
Test size: 9592
```

```
class NERDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return {
            "input_ids": self.X[idx],
            "labels": self.y[idx]
        }

train_loader = DataLoader(NERDataset(X_train, y_train),
                          batch_size=32,
                          shuffle=True)

test_loader = DataLoader(NERDataset(X_test, y_test),
                         batch_size=32,
                         shuffle=False)

print("Train batches:", len(train_loader))
print("Test batches:", len(test_loader))
```

```
Train batches: 1199
Test batches: 300
```

```
class BiLSTMTagger(nn.Module):

    def __init__(self, vocab_size, tagset_size, embedding_dim=100, hidden_dim=128):
        super(BiLSTMTagger, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)

        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            batch_first=True,
            bidirectional=True
        )

        self.fc = nn.Linear(hidden_dim * 2, tagset_size)

    def forward(self, input_ids):

        x = self.embedding(input_ids)

        lstm_out, _ = self.lstm(x)

        outputs = self.fc(lstm_out)

        return outputs
```

```
model = BiLSTMTagger(vocab_size, num_tags).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

print(model)
```

```
BiLSTMTagger(
  (embedding): Embedding(35179, 100, padding_idx=0)
  (lstm): LSTM(100, 128, batch_first=True, bidirectional=True)
  (fc): Linear(in_features=256, out_features=17, bias=True)
)
```

```
def train_model(model, train_loader, test_loader, loss_fn, optimizer, epochs=3):

    train_losses = []
    val_losses = []

    for epoch in range(epochs):

        model.train()
        total_loss = 0

        for batch in train_loader:

            input_ids = batch["input_ids"].to(device)
            labels = batch["labels"].to(device)

            optimizer.zero_grad()

            outputs = model(input_ids)

            loss = loss_fn(
                outputs.view(-1, outputs.shape[-1]),
                labels.view(-1)
            )

            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        train_loss = total_loss / len(train_loader)
        train_losses.append(train_loss)

        model.eval()
        val_loss = 0

        with torch.no_grad():
            for batch in test_loader:

                input_ids = batch["input_ids"].to(device)
```

```
                labels = batch["labels"].to(device)

                outputs = model(input_ids)

                loss = loss_fn(
                    outputs.view(-1, outputs.shape[-1]),
                    labels.view(-1)
                )

                val_loss += loss.item()

        val_loss = val_loss / len(test_loader)
        val_losses.append(val_loss)

        print(f"Epoch {epoch+1}/{epochs}")
        print("Train Loss:", train_loss)
        print("Val Loss:", val_loss)

    return train_losses, val_losses
```

```
train_losses, val_losses = train_model(
    model,
    train_loader,
    test_loader,
    loss_fn,
    optimizer,
    epochs=3
)
```

```
Epoch 1/3
Train Loss: 0.16618019841095624
Val Loss: 0.08310042310506105
Epoch 2/3
Train Loss: 0.06455876266085127
Val Loss: 0.06291940618306398
Epoch 3/3
Train Loss: 0.04644352447915763
Val Loss: 0.055538112024466194
```

```
def evaluate_model(model, test_loader):

    model.eval()

    true_tags = []
    pred_tags = []

    with torch.no_grad():
        for batch in test_loader:

            input_ids = batch["input_ids"].to(device)
            labels = batch["labels"].to(device)

            outputs = model(input_ids)
            preds = torch.argmax(outputs, dim=-1)

            for i in range(labels.shape[0]):
                for j in range(labels.shape[1]):
                    if labels[i][j].item() != tag2idx["O"]:
                        true_tags.append(idx2tag[labels[i][j].item()])
                        pred_tags.append(idx2tag[preds[i][j].item()])

    print("Name: Kukkadapu Charan Tej")
    print("Register Number: 212224040167")
    print(classification_report(true_tags, pred_tags))
```

```
evaluate_model(model, test_loader)
```

```
Name: Kukkadapu Charan Tej
Register Number: 212224040167
              precision    recall  f1-score   support

       B-art       0.00      0.00      0.00        94
       B-eve       0.79      0.21      0.34        70
       B-geo       0.88      0.86      0.87      7556
       B-gpe       0.95      0.93      0.94      3141
       B-nat       0.75      0.07      0.14        40
       B-org       0.82      0.66      0.73      4149
       B-per       0.84      0.75      0.79      3400
       B-tim       0.95      0.85      0.90      4074
       I-art       0.00      0.00      0.00        84
       I-eve       0.83      0.08      0.14        65
       I-geo       0.87      0.71      0.78      1462
       I-gpe       0.93      0.39      0.55        33
```

```
       I-nat       0.00      0.00      0.00        13
       I-org       0.84      0.73      0.78      3393
       I-per       0.85      0.84      0.84      3406
       I-tim       0.87      0.77      0.82      1251
           O       0.00      0.00      0.00         0

    accuracy                           0.79     32231
   macro avg       0.66      0.46      0.51     32231
weighted avg       0.87      0.79      0.83     32231

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defir
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defir
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defir
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

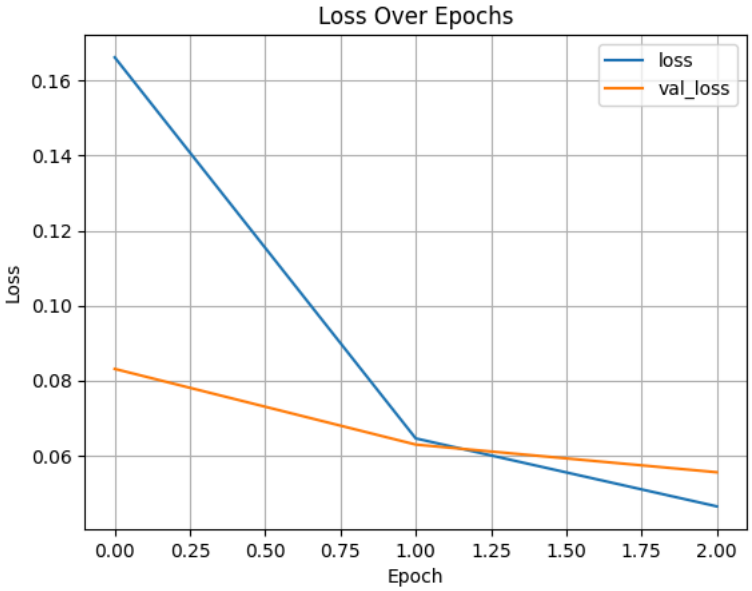
```
print("Name: Kukkadapu Charan Tej")
print("Register Number: 212224040167")

history_df = pd.DataFrame({
    "loss": train_losses,
    "val_loss": val_losses
})

history_df.plot(title="Loss Over Epochs")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

```
Name: Kukkadapu Charan Tej
Register Number: 212224040167
```




```
i = 10

model.eval()

sample = X_test[i].unsqueeze(0).to(device)
output = model(sample)

preds = torch.argmax(output, dim=-1).squeeze().cpu().numpy()

print("Name: Kukkadapu Charan Tej")
print("Register Number: 212224040167")

print("{:<15} {:<10} {}".format("Word","True","Pred"))
print("-"*40)

for w_id,true_tag,pred_tag in zip(X_test[i],y_test[i],preds):

    if w_id.item()!=word2idx[PAD_WORD]:
```

```
        word = words[w_id.item()-1]
        true_label = tags[true_tag.item()]
        pred_label = tags[pred_tag]

        print(f"{word:<15} {true_label:<10} {pred_label}")
```

```
Name: Kukkadapu Charan Tej
Register Number: 212224040167
Word            True       Pred
----------------------------------------
Officials       O          O
estimate        O          O
that            O          O
some            O          O
5,00,000        O          O
people          O          O
living          O          O
in              O          O
Latin           O          O
American        B-gpe      B-gpe
countries       O          O
such            O          O
as              O          O
Cuba            B-geo      B-geo
,               O          O
Argentina       B-geo      B-geo
,               O          O
Mexico          B-geo      B-geo
,               O          O
Venezuela       B-geo      B-geo
,               O          O
Chile           B-geo      B-geo
and             O          O
Uruguay         B-geo      B-geo
are             O          O
eligible        O          O
for             O          O
citizenship     O          O
.               O          O
```